

A dashboard for a project named **CareerConnect** should act as the central hub where users can manage their professional journey. Since your project connects job seekers with opportunities, the dashboard needs to show personalized, actionable data the moment a user logs in.

Here are the most common features and structures for this kind of dashboard, depending on who is logging in:

1. Job Seeker Dashboard (Standard User)

If the user is looking for a job, their dashboard should focus on tracking applications and finding new roles.

- **Application Tracker Cards:** Small visual panels showing stats like *Applied*, *Interviewing*, *Offered*, or *Rejected*.
- **Recommended Jobs:** A feed of open positions tailored to their profile name or skills.
- **Resume/Profile Completeness Bar:** A progress bar showing how much profile data they have filled out (e.g., "75% complete").
- **Recent Activity Log:** A timeline showing recent updates, like *"Recruiter viewed your profile 2 hours ago"*.

2. Recruiter / Employer Dashboard (Alternative User)

If you decide to let companies sign up later, their dashboard would focus on hiring metrics.

- **Job Postings Manager:** A table showing jobs they have listed and whether they are active or closed.
- **Applicant Summary Counts:** Quick numbers showing total applicants per job posting (e.g., "Software Engineer - 14 applicants").
- **Interview Scheduler:** A small calendar view or list of upcoming candidate interviews.

Dashboard Page Structure Overview

Component	What it displays	Purpose
Top Navigation Bar	User profile icon, notifications bell, and a "Log Out" button.	Keeps the session secure and accessible.
Sidebar Menu	Links to <i>Dashboard</i> , <i>Profile</i> , <i>Saved Jobs</i> , and <i>Messages</i> .	Allows easy navigation without reloading the app structure.
Main Content Grid	The metric blocks (stats) and	Gives an immediate summary

Component	What it displays	Purpose
	tables for active applications.	of their status.

Practical Starting Point

To get this working in your Express app right now, you can create a simple **dashboard.html** file inside your Pages folder.

Then, you can set up a route in your server.js file to serve it:

```
// Route for Dashboard page
app.get('/dashboard', (req, res) => {
  res.sendFile(path.join(__dirname, '..', 'Pages', 'dashboard.html'));
});
```

Later on, once we add user sessions, we will make sure people can only view this page if they are actually logged in!

Would you like to build a dashboard for a **Job Seeker** or a **Recruiter**, or do you want to start by creating a simple **universal layout** that works for both?